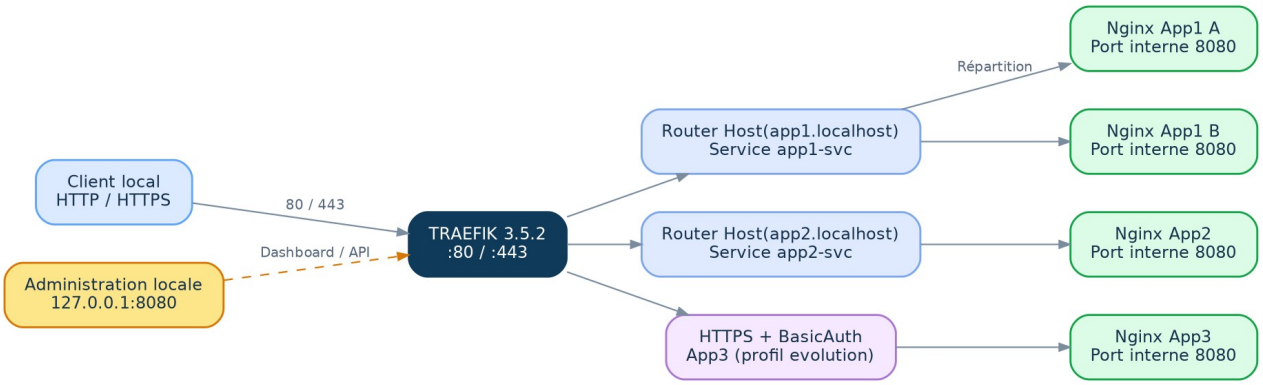
RAPPORT TECHNIQUE | DEVOPS & CYBERSECURITE

TP5 - Reverse Proxy et Load Balancing avec Traefik

DevTech Solutions • Docker Compose • Nginx • HTTPS et sécurité



Architecture du laboratoire : une entrée Traefik, trois applications, deux instances App1.

Données de remise	Détails
Établissement / équipe	À renseigner
Auteur(s)	À renseigner
Date	6 novembre 2026 (kit documentaire)
Périmètre	5 exercices, 21 questions, cartographie de sécurité, Docker Compose, tests
Statut technique	44 tests statiques réussis ; certificat/BCrypt vérifiés ; Docker non exécuté ici

Document de laboratoire. Aucun succès Docker, routage, TLS de bout en bout, scan Trivy ou accès distant n'est affirmé sans preuve réelle.

1. Objectif et choix d'architecture

Le sujet demande de remplacer plusieurs ports applicatifs par un point d'entrée unique Traefik et de démontrer le routage HTTP/HTTPS, la découverte Docker, la répartition entre deux backends, puis l'authentification et les protections de sécurité.

Le système livré comprend Traefik 3.5.2 et quatre conteneurs Nginx : App1 A, App1 B, App2 et App3 (profil evolution). App1 A/B répondent au même nom app1.localhost et partagent un service logique ; App2 répond sur app2.localhost ; App3 est protégée en HTTPS sur app3.localhost.

Point de sécurité central : aucune application Nginx ne publie de port sur l'hôte ; seuls les ports de Traefik 80, 443 et 8080 sont liés à 127.0.0.1.

Architecture technique

Composant	Fonction	Port / exposition
Traefik	Reverse proxy, router, TLS, dashboard/API	Hôte 127.0.0.1:80 / :443 / :8080
App1 / App1B	Deux instances Nginx pour la répartition	Docker 8080, aucun port hôte
App2	Deuxième application indépendante	Docker 8080, aucun port hôte
App3 (profil evolution)	Troisième application avec BasicAuth	HTTPS via Traefik uniquement
Réseau frontend	Communication Traefik <-> services Docker	Réseau partagé, segmentation cible non appliquée
CA locale + htpasswd	Certificat local et hash BCrypt hors Git	Fichiers créés par scripts, jamais livrés

Concepts à expliquer

- EntryPoint : port d'entrée réseau (80 ou 443).
- Router : règle de correspondance, par exemple Host(app1.localhost).
- Service : backend logique contenant une ou plusieurs instances.
- Middleware : traitement transversal, par exemple BasicAuth, en-têtes de sécurité ou redirection HTTPS.
- Provider Docker : découverte des conteneurs autorisés par leurs labels.
- Provider File : charge le certificat TLS et les middlewares décrits en YAML.

2. Réalisation des cinq exercices

Exercice 1 — Déploiement initial

Le réseau frontend est créé par bootstrap.sh s'il n'existe pas. Traefik lit config/traefik.yaml et publie les ports 80, 443 et 8080 uniquement sur l'interface loopback. Le dashboard est accessible localement à l'adresse http://127.0.0.1:8080/dashboard/. Il doit être testé réellement avec Docker.

La différence essentielle : l'entrypoint reçoit la requête sur un port ; le router applique une règle Host et désigne un service.

Exercice 2 — Routage par nom d'hôte

App1 et App2 possèdent le label traefik.enable=true. Leurs routers distinguent app1.localhost et app2.localhost, bien qu'ils partagent le port 80 de Traefik. Le port Nginx interne est 8080 : l'image non privilégiée utilise ce port plutôt que 80, ce qui constitue un durcissement documenté par rapport à l'exemple du PDF.

Commandes : curl --noproxy "*" -H "Host: app1.localhost" http://127.0.0.1/ ; même test pour app2.localhost ; consulter /api/http/routers et /api/http/services.

Exercice 3 — Répartition App1 A/B

Deux conteneurs Nginx servent des réponses différentes : « App1 - Instance A » et « App1 - Instance B ». Les labels des deux conteneurs sont identiques. Le résultat attendu est un service app1-svc@docker comportant deux backends ; cette agrégation doit être confirmée par l'API Traefik et par vingt requêtes alternées.

Le routage choisit l'application ; le load balancing choisit une instance au sein de l'application. Les healthchecks aident à écarter une instance indisponible.

Exercice 4 — Sécurité du reverse proxy

Traefik est configuré avec exposedByDefault=false, et seul un label d'activation autorise une route Docker. Le service non-expose, lancé à part, sert à vérifier ce principe. Le dashboard/API n'est publié qu'en local. Aucune information d'authentification n'est placée dans les labels.

Il reste deux risques majeurs du laboratoire : api.insecure=true n'authentifie pas les utilisateurs locaux ; docker.sock:ro protège le montage mais ne limite pas les opérations de l'API Docker. Une solution de production nécessiterait un dashboard sécurisé et un proxy de socket filtré.

Exercice 5 — Évolution et HTTPS

App3 peut être activée avec docker compose --profile evolution up -d app3. Elle dispose d'un router HTTPS et d'un middleware BasicAuth lisant un fichier BCrypt exclu du dépôt. Le certificat couvre app1.localhost, app2.localhost et app3.localhost. La confiance est validée par curl --cacert, sans désactiver la vérification TLS.

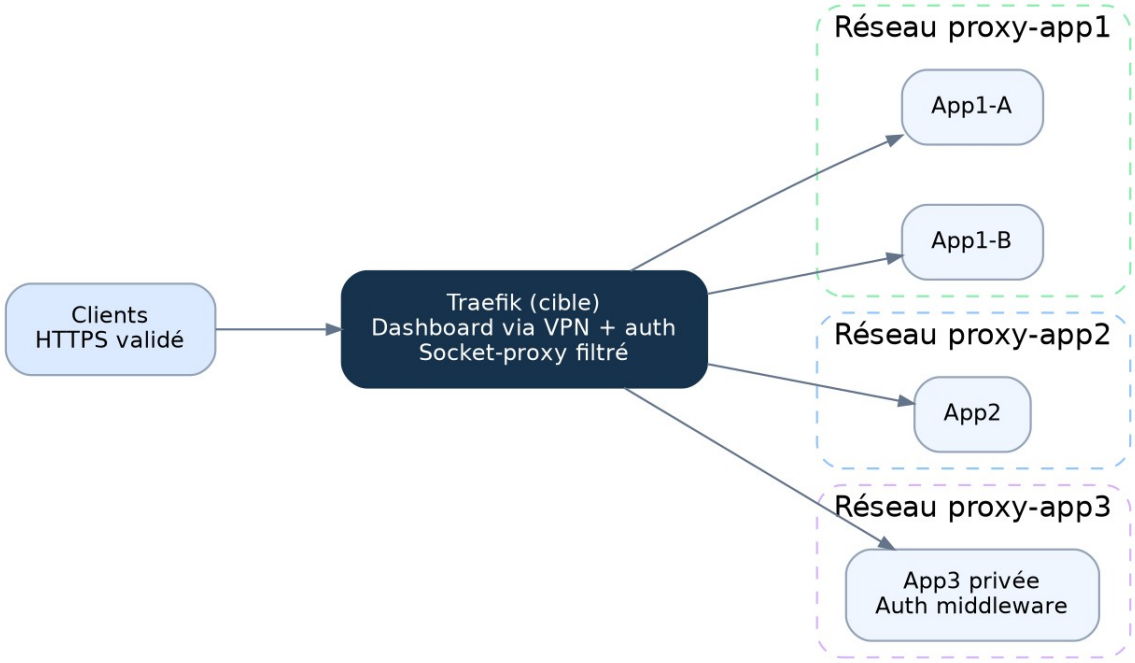
La variante cible prévoit une segmentation des réseaux applicatifs, l'authentification du dashboard, des healthchecks et une architecture multi-hôtes pour la disponibilité. Le réseau frontend partagé de ce laboratoire n'isole pas les applications les unes des autres.

3. Cartographie exploitable pour la sécurité

La section 20 du sujet demande de relier chaque composant, son exposition, le risque, la protection attendue et la preuve de validation. Le tableau de référence détaillé est disponible dans docs/ARCHITECTURE_SECURITE.md.

Zone	Risque	Protection	Preuve	P.
Entrée 80/443	Interception, accès illicite	HTTPS certifié, exposition limitée	curl --cacert et tests réseau	P1
Dashboard/API	Divulgation de topologie	Loopback au TP ; VPN/auth en prod	Accès local oui ; externe refusé	P1
API Docker	Compromission potentielle hôte	Socket-proxy filtré (cible prod)	Inspection mounts/endpoints	P1
App1/App1B	Panne, contournement	Aucun port hôte, healthcheck	inspect + test panne	P1
App2/App3	Exposition involontaire	Opt-in, auth et segmentation	404/401/200, réseau	P1
Configurations	Routage illégitime	Git/revue, secrets exclus	diff et contrôle secrets	P1
Certificats et hash	Fuite de clés, expiration	0600, TLS validé, rotation	stat, openssl verify	P1
Images et conteneurs	CVE, privilèges excessifs	Non-root, read_only, scanner	inspect, scan à exécuter	P1
Logs	Données sensibles, manque de traces	Rétention et accès limités	Échantillons expurgés	P2
Hôte et proxy	Point unique de panne	Redondance et restauration cible	Test panne multi-hôtes	P2

Cible de segmentation (proposition, non déployée)



Cible : Traefik raccordé aux réseaux applicatifs distincts ; backend App1 redondé.

Les deux instances App1 du laboratoire restent sur un seul hôte Docker : elles améliorent la répartition et peuvent tolérer l'arrêt d'un conteneur, mais ne résistent pas à une panne complète de cet hôte.

4. Vérification et statut réel

Les tests statiques ont été réellement exécutés dans l'environnement de préparation, sans moteur Docker. Ils inspectent le Compose, la configuration Traefik, les labels, l'absence de ports Nginx publiés, les protections non-root et read_only, les scripts, le HTTPS et la couverture des 21 questions.

Contrôle	Résultat établi	Limite
pytest local	44 tests statiques réussis	Ne démarre aucun conteneur
Bash scripts	Syntaxe validée par bash -n	Ne valide pas l'exécution réseau
Certificats local	CA/certificat signés ; SAN + openssl verify OK	Aucune connexion HTTPS Traefik réelle
BasicAuth	Hash BCrypt généré et vérifié ; fichiers en mode 600	Pas de requête Traefik 401/200 ici
docker compose config	Non exécuté dans cet environnement : Docker absent	À réaliser sur poste équipé
Traefik, Nginx, 2 backends	Non exécuté ici	Scripts d'intégration fournis
API/dashboard refus distant	Non exécuté ici	Requiert autre machine autorisée au test
Scan d'images Trivy	Non exécuté ici	À faire sur images effectivement utilisées

Procédure sur une machine disposant de Docker

./scripts/bootstrap.sh

./scripts/smoke-test.sh

./scripts/check-unlabeled.sh

docker compose --profile evolution up -d app3

TP5_TEST_ADMIN=1 ./scripts/smoke-test.sh

docker compose --profile evolution --profile audit down

Les preuves ne doivent être ajoutées à la remise qu'après un succès réellement constaté. La checklist fournit les commandes et le détail des refus attendus. Ne jamais capturer les mots de passe ou les clés privées.

5. Questions de compréhension corrigées

1. Qu'est-ce qu'un reverse proxy ?

Un serveur qui reçoit les requêtes des clients puis les transmet aux applications appropriées. Il centralise l'entrée du trafic et masque les adresses internes des backends.

2. Quelle est la différence entre un reverse proxy et un Load Balancer ?

Un reverse proxy sert d'intermédiaire et route les requêtes. Un load balancer répartit le trafic entre plusieurs instances d'un service. Traefik peut remplir les deux rôles.

3. Quel est le rôle de Traefik dans une architecture Docker ?

Traefik découvre les conteneurs autorisés, configure leurs routes à partir des labels et répartit les requêtes vers les bons services.

4. Quelle est la différence entre configuration statique et dynamique ?

La statique initialise Traefik (entrypoints, providers, API). La dynamique décrit les routers, services, middlewares et paramètres TLS des routes.

5. À quoi sert un provider ?

À fournir à Traefik des informations de configuration. Le provider Docker lit les labels des conteneurs ; le provider file charge la configuration d'un fichier.

6. Quel est le rôle d'un entrypoint ?

C'est l'adresse/port réseau par lequel entre le trafic, par exemple web sur le port 80 ou websecure sur 443.

7. Quelle différence entre un router et un service ?

Le router décide quelle règle correspond à la requête (Host(...)) ; le service représente le backend vers lequel envoyer cette requête.

8. Pourquoi utilise-t-on des labels Docker ?

Pour déclarer les routes, ports internes et middlewares directement dans la définition des conteneurs et permettre la découverte automatique par Traefik.

9. Quel est l'intérêt de `exposedByDefault: false` ?

Éviter qu'un conteneur quelconque devienne automatiquement accessible par Traefik. Seuls ceux explicitement étiquetés traefik.enable=true peuvent être routés.

10. Comment Traefik découvre-t-il automatiquement un nouveau conteneur ?

Le provider Docker surveille les événements/états du daemon Docker, lit les labels des conteneurs autorisés et actualise la configuration dynamique.

11. Quelle différence entre routage et Load Balancing ?

Le routage choisit l'application, le load balancing choisit l'une des instances disponibles de cette même application.

12. Pourquoi surveiller la santé des backends ?

Pour éviter d'envoyer des requêtes vers une instance indisponible et améliorer la continuité du service. Un healthcheck ne remplace pas un plan de haute disponibilité multi-hôtes.

13. Pourquoi protéger le dashboard ?

Il révèle la topologie, les routes, les services et leur configuration. Un accès non autorisé facilite la reconnaissance et pourrait augmenter les risques opérationnels.

14. Pourquoi un router TLS ne garantit-il pas à lui seul un certificat valide ?

Parce que tls=true ne garantit pas que le certificat soit signé par une autorité approuvée, qu'il corresponde au nom demandé ou qu'il soit encore valide. Il faut vérifier la chaîne de confiance.

15. Comment ajouter une instance backend sans changer le point d'entrée client ?

Ajouter un deuxième conteneur au même service logique de Traefik (ou augmenter ses réplicas) ; le client conserve app1.localhost et son port habituel.

16. Pourquoi `Host(...)` ne constitue-t-il pas une authentification ?

Parce que n'importe quel client peut envoyer un en-tête Host. La règle sélectionne un router, elle ne prouve pas l'identité ni les permissions du client.

17. Pourquoi `docker.sock:ro` ne garantit-il pas une API Docker en lecture seule ?

Parce que :ro s'applique au montage du fichier socket, pas aux méthodes HTTP autorisées sur l'API Docker. Un client capable d'utiliser le socket peut toujours envoyer des opérations sensibles selon ses droits.

18. Pourquoi protéger l'API autant que le dashboard ?

L'API expose elle aussi des informations internes de routage et d'état. Limiter seulement l'interface visuelle ne suffit pas ; les endpoints JSON doivent recevoir les mêmes protections.

19. Quelle différence entre réseau partagé et segmentation réseau ?

Un réseau partagé permet aux conteneurs de se joindre. La segmentation répartit les composants en réseaux séparés et limite les communications aux flux nécessaires afin de réduire les mouvements latéraux.

20. Pourquoi deux instances sur le même hôte ne suffisent-elles pas à la haute disponibilité ?

Parce qu'une panne de l'hôte Docker, de son réseau ou du proxy peut arrêter simultanément les deux instances. La haute disponibilité exige d'éliminer les points de défaillance communs.

21. Quelles preuves permettent de vérifier qu'un contrôle de sécurité fonctionne ?

Les configurations seules ne suffisent pas : montrer des tests et résultats réels. Par exemple API/dashboard refusés depuis une machine non autorisée, aucun port Nginx publié, 404 hôte inconnu, conteneur non étiqueté absent des routes, certificat TLS validé sans -k, BasicAuth 401/200, tests de santé et traces expurgées.

6. Limites, recommandations et sources

- Docker n’étant pas disponible dans l’environnement de création, aucune requête HTTP, HTTPS, API/dashboard ou démonstration de load balancing réelle ne peut être revendiquée ici.
- Le port 8080 en mode api.insecure=true n’est protégé que par une liaison à loopback dans le laboratoire ; en production, supprimer ce mode et ajouter des protections d’administration.
- Le socket Docker monté en lecture seule reste un point de confiance majeur ; préférer un proxy de socket avec endpoints limités.
- Le certificat de laboratoire n’est pas une certification publique ; la confiance est donnée explicitement à curl par une CA locale contrôlée.
- Le réseau frontend commun n’est pas une segmentation, et deux instances sur un hôte ne constituent pas une haute disponibilité multi-hôtes.
- Les images ne sont pas verrouillées par digest et aucun scan Trivy n’a été exécuté dans ce kit ; ces mesures restent à réaliser pour un autre niveau d’assurance.

Références : sujet TP5 DevTech Solutions (26 pages, sections 18 à 20) ; documentation officielle Traefik 3.5 Docker provider ; Traefik File Provider/TLS ; Traefik BasicAuth ; documentation NGINX Unprivileged. Liens précis disponibles dans le README du projet.

Rendu : sources Docker et YAML, scripts, HTML, tests statiques, correction des 5 exercices, réponses aux 21 questions, schémas SVG et rapport Word modifiable. Aucun secret généré n’est inclus dans l’archive.